4장. 데이터베이스와 ORM 사용하기

로컬 메모리 데이터 구조는 애플리케이션 재기동 시 데이터가 초기화된다는 치명적인 한계가 있습니다. 본 장에서는 가볍고 이식성이 뛰어난 관계형 데이터베이스인 **SQLite**를 연동하고, 파이썬 ORM 라이브러리의 업계 표준인 **SQLAlchemy**를 활용해 데이터의 영속성을 처리하는 과정을 상세히 실습합니다.

4.1 SQLite와 ORM의 이해

SQLite는 서버 구동이 필요 없이 단일 파일 형태로 동작하는 가볍고 신속한 관계형 DB 엔진입니다. ORM(Object-Relational Mapping)은 객체 지향 언어의 **객체(클래스)**와 관계형 데이터베이스의 **테이블**을 자동으로 매핑시켜 주어 개발자가 SQL 질의문 없이도 코드로 쿼리를 실행할 수 있게 해줍니다.

4.2 SQLAlchemy 커넥션 설정

먼저 데이터베이스 연결 및 세션 생성을 관리할 `database.py` 파일을 생성합니다.

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# SQLite 로컬 파일 경로 정의 (프로젝트 내 todos.db 생성)
SQLALCHEMY_DATABASE_URL = "sqlite:///./todos.db"

# SQLite는 멀티스레드 접근 시 기본적으로 커넥션을 제어하므로 check_same_thread 옵션 필요
engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)

# 데이터베이스 세션을 가동할 클래스 생성
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# ORM 모델들이 상속받아 정의할 Base 클래스
Base = declarative_base()
```

4.3 ORM 데이터베이스 모델 설계

SQLite 데이터베이스 내 테이블과 1대1 매핑될 파이썬 클래스를 정의합니다. `Base` 를 상속합니다.

```
# models.py
from sqlalchemy import Column, Integer, String, Boolean
from database import Base

class TodoModel(Base):
    __tablename__ = "todos"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(50), nullable=False)
    content = Column(String(200))
    is_done = Column(Boolean, default=False)
```

4.4 데이터베이스 세션 주입 및 테이블 자동 생성

FastAPI 구동 시 SQLite 테이블을 생성하고, 의존성 주입(Depends) 기능을 사용해 API 요청마다 세션을 열고 닫는 과정을 구조화합니다. 최신 파이썬의 파이프(|) 타입 힌트를 적극 사용합니다.

```
# main.py
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from database import SessionLocal, engine, Base
from models import TodoModel
from pydantic import BaseModel

# 테이블 초기 자동 생성
Base.metadata.create_all(bind=engine)

app = FastAPI()

# 💉 Pydantic 스키마 정의 (데이터 검증용)
class TodoBase(BaseModel):
    title: str
    content: str | None = None
    is_done: bool = False

class TodoResponse(TodoBase):
    id: int

    class Config:
        from_attributes = True  # SQLAlchemy 객체를 Pydantic으로 자동 파싱 (Pydantic v2 표준)

# 🔑 DB 세션을 얻기 위한 의존성 주입 함수
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close() # API 요청 완료 후 커넥션 풀 반환 및 자동 세션 닫기
```

4.5 DB 연동 CRUD API 완성

의존성이 주입된 `db: Session` 을 바탕으로 SQLite 테이블에 데이터를 직접 삽입, 조회, 갱신하는 코드를 작성합니다.

```
@app.post("/todos", response_model=TodoResponse, status_code=201)
def create_todo(todo: TodoBase, db: Session = Depends(get_db)) -> TodoModel:
    db_todo = TodoModel(
        title=todo.title,
        content=todo.content,
        is_done=todo.is_done
    )
    db.add(db_todo)
    db.commit()      # 데이터베이스 반영
    db.refresh(db_todo) # 생성된 자동증가 ID 값 등을 로드
    return db_todo

@app.get("/todos", response_model=list[TodoResponse])
def get_todos(db: Session = Depends(get_db)) -> list[TodoModel]:
    return db.query(TodoModel).all()

@app.get("/todos/{todo_id}", response_model=TodoResponse)
def get_todo(todo_id: int, db: Session = Depends(get_db)) -> TodoModel:
    db_todo = db.query(TodoModel).filter(TodoModel.id == todo_id).first()
    if not db_todo:
        raise HTTPException(status_code=404, detail="할 일을 찾을 수 없습니다.")
    return db_todo

@app.put("/todos/{todo_id}", response_model=TodoResponse)
def update_todo(todo_id: int, todo: TodoBase, db: Session = Depends(get_db)) -> TodoModel:
    db_todo = db.query(TodoModel).filter(TodoModel.id == todo_id).first()
    if not db_todo:
        raise HTTPException(status_code=404, detail="수정할 항목이 없습니다.")
    db_todo.title = todo.title
    db_todo.content = todo.content
    db_todo.is_done = todo.is_done
    db.commit()
    db.refresh(db_todo)
    return db_todo
```